Experiment No.	4
Experiment Title	Binary Classification using Logistic Regression and SVM
GitHub Repository	https://github.com/Dharika-2006/ML-LAB-ASSIGMENTS
Student Name	Dharika Shree K
Register Number	3122247001014
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	02.08.26

1 Objective

To build and compare two binary classifiers – Logistic Regression and Support Vector Machine (SVM) – on the Spambase email dataset, tune their hyperparameters using GridSearchCV, and evaluate them using accuracy, precision, recall, F1-score, confusion matrices, ROC curves, and 5-fold cross-validation.

2 Problem Statement

Problem: Classify emails as spam or ham using Logistic Regression and SVM, and compare which one works better on this dataset also testing SVM across four different kernels (linear, poly, rbf, sigmoid).

Input: The Spambase dataset , 57 numeric features describing word/character frequencies and capital-letter usage per email.

Output: Trained and tuned Logistic Regression and SVM models, kernel-wise SVM comparison, confusion matrices, ROC curves, cross-validation scores, and a final accuracy/precision/recall/F1 comparison between the two algorithms.

3 Dataset Description

Dataset Name	Spambase dataset
Dataset Source	Kaggle
Number of Samples	4601
Number of Features	57
Number of Classes	2
Missing Values	None
Train-Test Split	80% training and 20% testing

4 Exploratory Data Analysis

- Dataset overview
- Statistical summary
- Missing value analysis
- Class distribution

- Correlation matrix
- Feature distribution

Figure: Class Distribution

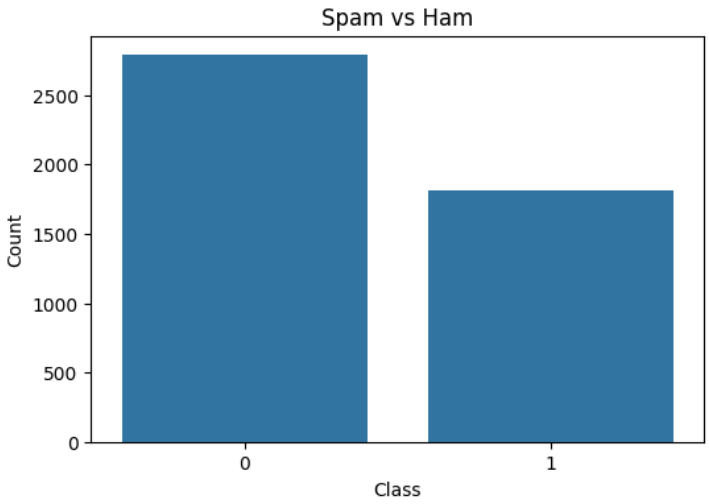

Figure 1: Spam vs Ham class counts

Inference: About 60% of the emails are ham and 40% are spam (mean of the `class` column is 0.394), so it's not a perfectly balanced dataset but not extreme either. This is exactly why the train-test split was stratified – to keep that same ratio in both splits – and why we're looking at precision/recall/F1 and not just accuracy.

Figure: Statistical Summary

Statistical Summary

	word_freq_make	word_freq_address	word_freq_all	word_freq_3d
count	4601.000000	4601.000000	4601.000000	4601.000000
mean	0.104553	0.213015	0.280656	0.065425
std	0.305358	1.290575	0.504143	1.395151
min	0.000000	0.000000	0.000000	0.000000
25%	0.000000	0.000000	0.000000	0.000000
50%	0.000000	0.000000	0.000000	0.000000
75%	0.000000	0.000000	0.420000	0.000000
max	4.540000	14.280000	5.100000	42.810000

	word_freq_our	word_freq_over	word_freq_remove	word_freq_internet
count	4601.000000	4601.000000	4601.000000	4601.000000
mean	0.312223	0.095901	0.114208	0.105295
std	0.672513	0.273824	0.391441	0.401071
min	0.000000	0.000000	0.000000	0.000000
25%	0.000000	0.000000	0.000000	0.000000
50%	0.000000	0.000000	0.000000	0.000000
75%	0.380000	0.000000	0.000000	0.000000
max	10.000000	5.880000	7.270000	11.110000

	word_freq_order	word_freq_mail	...	char_freq_%3B	char_freq_%28
count	4601.000000	4601.000000	...	4601.000000	4601.000000
mean	0.090067	0.239413	...	0.038575	0.139030
std	0.278616	0.644755	...	0.243471	0.270355
min	0.000000	0.000000	...	0.000000	0.000000
25%	0.000000	0.000000	...	0.000000	0.000000
50%	0.000000	0.000000	...	0.000000	0.065000
75%	0.000000	0.160000	...	0.000000	0.188000
max	5.260000	18.180000	...	4.385000	9.752000

	char_freq_%5B	char_freq_%21	char_freq_%24	char_freq_%23
count	4601.000000	4601.000000	4601.000000	4601.000000
mean	0.016976	0.269071	0.075811	0.044238
std	0.109394	0.815672	0.245882	0.429342
min	0.000000	0.000000	0.000000	0.000000
25%	0.000000	0.000000	0.000000	0.000000
50%	0.000000	0.000000	0.000000	0.000000
75%	0.000000	0.315000	0.052000	0.000000
max	4.081000	32.478000	6.003000	19.829000

Inference: The `describe()` output backs up what the histograms show – for most features the 25th, 50th and 75th percentiles are all 0, meaning over half the emails have a

frequency of 0 for that word, while the mean is pulled up by a handful of high-frequency outliers. No missing values anywhere, so no imputation was needed here (unlike the earlier experiments).

Figure: Correlation Matrix

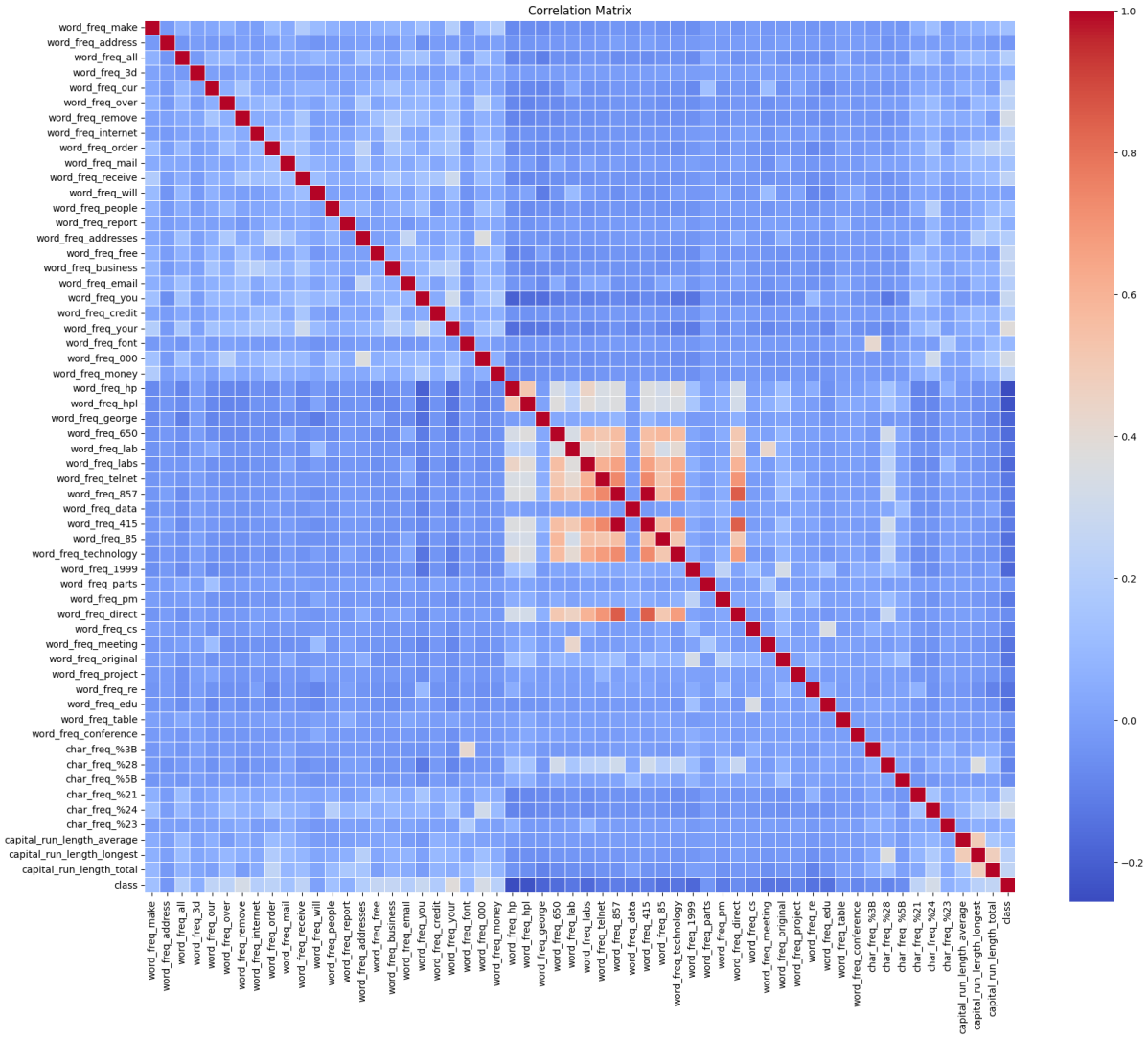

Figure 2: Correlation heatmap of the 57 numeric features

Inference: Most word-frequency features are only weakly correlated with each other, which works in favour of both models here – Logistic Regression doesn't get too confused by multicollinearity, and SVM can find a clean separating boundary. A handful of related features (like the three capital-run-length stats, or spam-flavoured words like "free", "money", "credit") do move together and are the ones doing most of the heavy lifting in classification.

5 Data Preprocessing

- **Missing value handling:** Not needed – the dataset had zero missing values.

- **Encoding:** Not needed – all 57 features are already numeric (word/character frequencies and capital-run-length stats).
- **Feature scaling:** All features were standardized using `StandardScaler` (mean 0, unit variance) before training, since both Logistic Regression and SVM are distance/scale sensitive.
- **Train-test split:** 80/20 split using `train_test_split` with `stratify=y` and `random_state=42`, so the spam/ham ratio stays consistent in both sets.
- **Feature engineering:** None – the original 57 features were used as-is.

6 Machine Learning Algorithm

Logistic Regression

Working principle: Models the log-odds of the positive class as a linear combination of the input features, then squashes it through the sigmoid function to get a probability.

Mathematical formulation:

$$P(y=1 \mid x) = \sigma(w^T x+b) = \frac{1}{1+e^{-(w^T x+b)}}, \qquad \min_{w,b} -\sum_i \left[y_i \log \hat{y}_i + (1-y_i)\log(1-\hat{y}_i)\right] + \lambda\,\Omega(w)$$

where $\Omega(w)$ is the L1 or L2 penalty term (tuned via the `penalty` hyperparameter in this experiment).

Advantages: Fast, interpretable coefficients, works well when classes are roughly linearly separable, cheap to train (0.04s in this experiment).

Limitations: Struggles with non-linear decision boundaries, sensitive to outliers and unscaled features.

Support Vector Machine (SVM)

Working principle: Finds the hyperplane that maximizes the margin between the two classes; the kernel trick lets it draw non-linear boundaries by implicitly mapping data into a higher-dimensional space.

Mathematical formulation:

$$\min_{w,b} \ \tfrac{1}{2}\|w\|^2 + C\sum_i \xi_i \quad \text{s.t. } y_i(w^T\phi(x_i)+b) \geq 1-\xi_i$$

Kernels tested: linear $K(x,x') = x^T x'$, polynomial, RBF $K(x,x') = e^{-\gamma\|x-x'\|^2}$, and sigmoid.

Advantages: Handles non-linear boundaries well via kernels, generally robust to overfitting when C is tuned properly, gave the best precision of all models tested (0.928 with the tuned RBF kernel).

Limitations: Slower to train than Logistic Regression (2s for the linear kernel vs. 0.04s for Logistic Regression), more hyperparameters to tune (C, γ, kernel choice), and the polynomial kernel here badly underperformed (recall of just 0.46), showing how much kernel choice matters.

7 Experimental Setup

Python Version	3 (exact version not captured; run on Google Colab's Python 3 runtime)
Scikit-Learn Version	As installed in the Colab environment (verify with `sklearn.__version__`)
IDE	Google Colaboratory (Jupyter-based notebook)
Operating System	Linux (Google Colab backend)
Hardware	Google Colab-provided CPU runtime

8 Hyperparameter Selection

Parameter	Value
Train-Test Split	80:20, stratified, `random_state=42`
Cross-Validation	5-fold, scoring = accuracy
Logistic Regression – search space	$penalty \in \{l1, l2\}$, $C \in \{0.01, 0.1, 1, 10, 100\}$, $solver \in \{liblinear,$
Logistic Regression – best params	$C = 1$, $penalty = l1$, $solver = saga$ (Best CV Accuracy = 0.923
SVM – search space	$kernel \in \{linear, rbf\}$, $C \in \{1, 10\}$, $gamma = scale$
SVM – best params	$C = 1$, $gamma = scale$, $kernel = rbf$ (Best CV Accuracy = 0.9

9 Results

Model	Accuracy	Precision	Recall	F1-score
Logistic Regression (tuned)	0.9294	0.9209	0.8981	0.9093
SVM (tuned, RBF kernel)	0.9273	0.9277	0.8843	0.9055

Logistic Regression edges out SVM slightly on accuracy, recall and F1, while tuned SVM has a marginally better precision. Honestly, the two models land within about 0.2% of each other on accuracy – close enough that either would be a reasonable choice here.

10 Result Visualization

Include all graphs generated during the experiment.

1. Correlation Matrix

See Section 4 – weak correlations between most features, a few standout groups (capital-run-length stats, spam-flavoured words) driving most of the signal.

2. Class Distribution

See Section 4 – roughly 60/40 ham-to-spam split.

3. Confusion Matrix

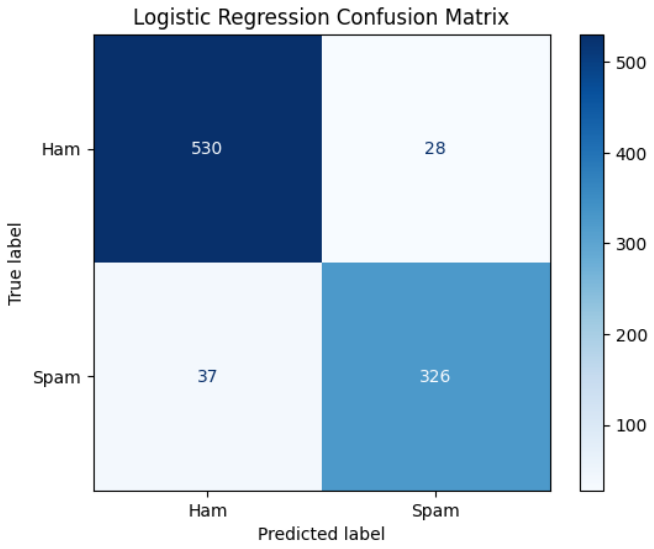

Figure 3: Confusion Matrix – Logistic Regression

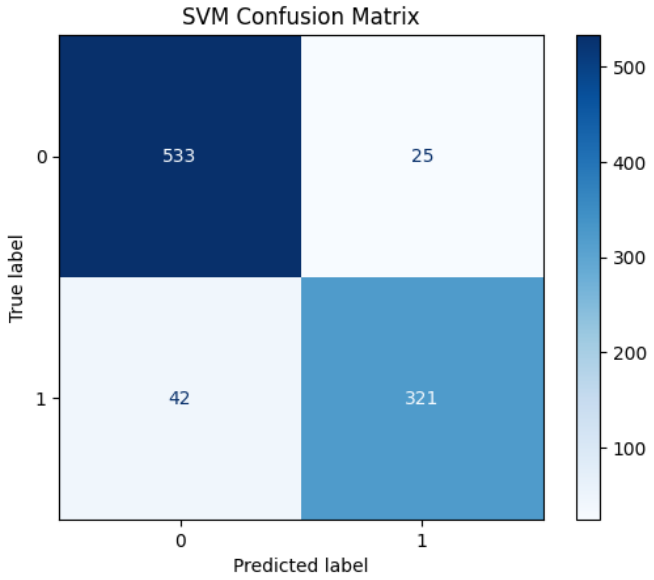

Figure 4: Confusion Matrix – SVM (RBF kernel)

Inference: Both models make more false-negative errors (spam missed as ham) than false-positive ones, which lines up with recall (0.898 for LR, 0.884 for SVM) being a touch lower than precision for both. Logistic Regression catches slightly more spam overall, while SVM is a bit more cautious before calling something spam – which is why its precision is marginally higher.

4. ROC Curve

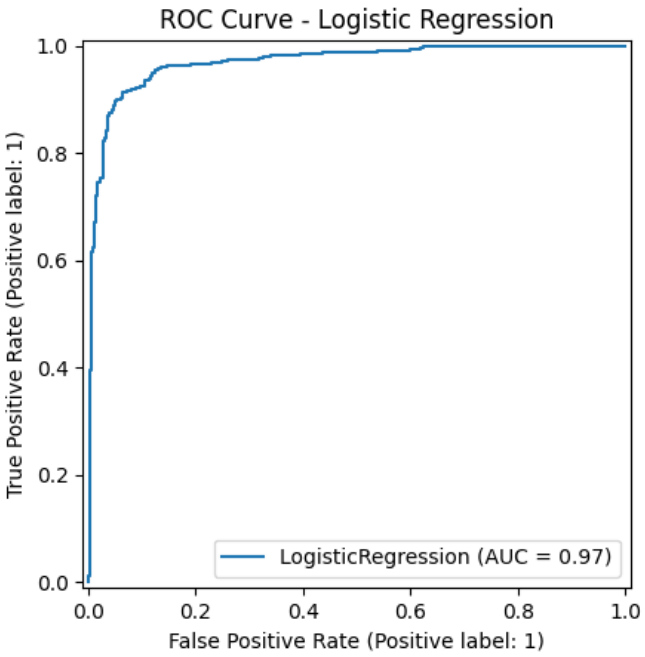

Figure 5: ROC Curve – Logistic Regression

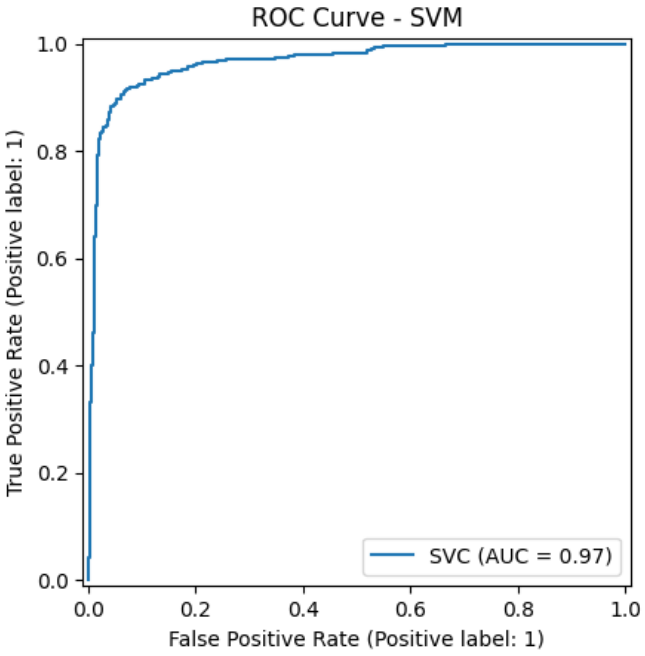

Figure 6: ROC Curve – SVM

Inference: Both ROC curves hug the top-left corner pretty tightly, which fits with both models scoring above 0.92 accuracy. There's no dramatic separation between the two curves – consistent with how close their F1-scores are overall.

5. Precision-Recall Curve

Not generated separately in the notebook (ROC curves were used instead), but the precision/recall numbers already reported tell the same story: SVM trades a bit of recall for precision compared to Logistic Regression.

6. SVM Kernel Comparison

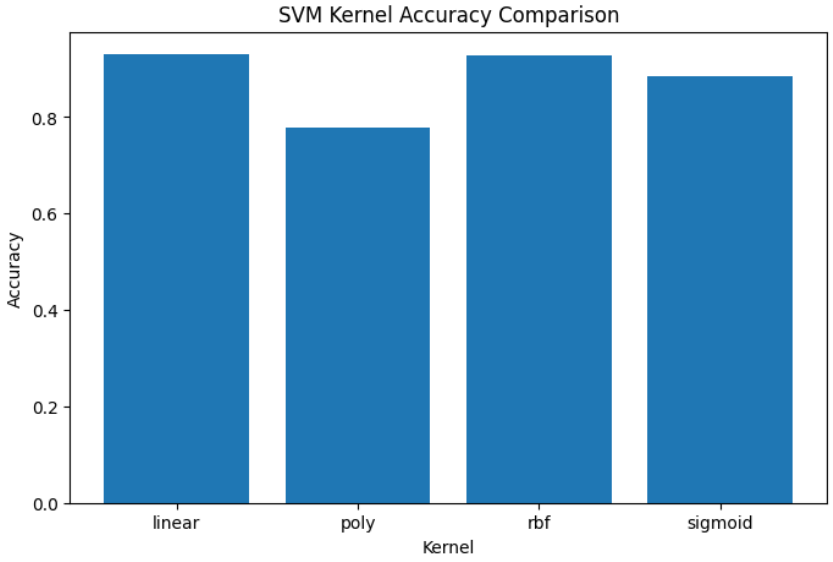

Figure 7: Accuracy across the four SVM kernels

Inference: Linear (0.9305) and RBF (0.9273) kernels are clearly the strongest performers, sigmoid trails behind at 0.884, and poly is by far the worst (0.780 accuracy, and only 0.46 recall) – it's basically missing more than half the spam emails. This shows the data is close to linearly separable, so the extra flexibility of poly/sigmoid kernels doesn't help and, if anything, hurts here.

7. Learning Curve

Not generated for this experiment (GridSearchCV was used instead of learning curves for tuning diagnostics).

8. Feature Importance

Not explicitly plotted, but Logistic Regression's coefficients (via the L1 penalty selected by GridSearchCV) would naturally zero out less useful features, and the correlation matrix already hints that capital-letter usage and spam-associated words are the most predictive features.

11 Discussion

Both models did well here – north of 92% accuracy each – which isn't too surprising given Spambase is a fairly clean, well-studied dataset once scaled properly. Logistic

Regression came out very slightly ahead on accuracy/recall/F1, and it's also way cheaper to train (milliseconds vs. seconds for SVM), so honestly it's the more practical pick if training time or interpretability matters. The one thing worth flagging: GridSearchCV for Logistic Regression took 581 seconds (almost 10 minutes!) because of the `saga` solver combined with L1 – that's a big cost for landing on basically the same performance as the untuned baseline. On the SVM side, kernel choice mattered a lot more than expected: poly kernel completely fell apart on recall (0.46), showing that not every kernel suits every dataset, and rbf/linear were clearly the better bets. Cross-validation results (SVM averaging 0.923, Logistic Regression 0.911 across 5 folds) also back up that both models are fairly stable, though fold 5 dipped noticeably for both, which is probably just a harder split of the data. If I were pushing this further, I'd try feature selection to cut down the 57 features, maybe try a Random Forest for comparison, and also look at why fold 5 underperformed.

12 Conclusion

This experiment compared Logistic Regression and SVM (across four kernels) for spam classification on the Spambase dataset. After hyperparameter tuning, Logistic Regression (C=1, L1 penalty, saga solver) slightly outperformed the best SVM (RBF kernel, C=1) on accuracy, recall and F1, while SVM edged ahead on precision. Given how close the two are and how much faster Logistic Regression is to train, it comes across as the more practical choice for this dataset – though SVM with the right kernel (linear or RBF, definitely not poly) is a solid alternative. Overall, the experiment reinforced that simpler, well-regularized linear models can hold their own against more complex kernel methods when the underlying data is close to linearly separable, as Spambase turned out to be.

13 References

1. Scikit-learn developers,Logistic Regression,scikit-learn documentation: https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

2. Scikit-learn developers,Support Vector Machines,scikit-learn documentation: https://scikit-learn.org/stable/modules/svm.html

3. Kaggle,Spambase dataset,Kaggle Datasets: https://www.kaggle.com/datasets